

A Note on Space Filling Curves

A. J. COLE

*Departmental of Computational Science, North Haugh University of St Andrews,
St Andrews, KY16 9SX. Scotland*

SUMMARY

Recursive procedures for drawing Peano, Hilbert and Sierpinski polygons are discussed. In each case the starting figure is a single point and the procedures clearly reflect the underlying geometry of the construction of the polygons. No assignment is required in any of the procedures.

KEY WORDS Graphics Peano Hilbert Sierpinski

INTRODUCTION

Peano¹ introduced the idea of a space filling curve in the sense of a continuous mapping of the line segment $(0,1)$ on to the unit square and was closely followed by Hilbert² and Sierpinski.³ Moore⁴ discussed the relationship between the geometric and analytic interpretation of such curves.

Although Peano's original curve established a direct relationship between the points of the line segment $(0,1)$ and the unit square, most of the subsequent work uses the idea introduced by Hilbert of a space filling curve as a limit of polygonal approximations. Figures 1, 2 and 6 represent the first few polygonal approximations to the Hilbert, Peano and Sierpinski curves, respectively.

In this paper we describe some simple recursive algorithms for drawing all of these approximations with the starting figure in each case being a single point. We also indicate how other curves may be derived by suitably modifying the algorithms.

A note in this journal⁵ described rather complicated Fortran subroutines for drawing Peano, Sierpinski and some other space filling curves. Wirth⁶ gives rather more understandable but lengthy Pascal procedures for drawing the Hilbert and Sierpinski curves.

Goldschlager⁷ gives some shorter recursive algorithms which draw the polygons as a sequence of line segments. The algorithm for drawing the Peano polygons is not given but would require nine calls of the recursive procedure together with eight additional line segment calls for each complete subpolygon. In addition the basic step length needs to be calculated separately in order to fit the final polygon into the available frame.

Witten and Wyvill⁸ also described algorithms for drawing space filling curves and some applications to texture and backdrop generation for computer graphics and animation.

The programming language used for our procedures is the Outline graphics system of S-algol.^{9,10} We only need a few of the ideas in this system to write all of our procedures.

OUTLINE GRAPHICS

The data type **pic** is a scalar data type with objects of type **pic** describing abstract pictures which can be drawn by the function

draw(*p*, *xmin*, *xmax*, *ymin*, *ymax*)

where *p* is the name of some object of type **pic** and *xmin*, *xmax*, *ymin*, *ymax*, are of type **real** and determine the current screen limits. The basic object of type **pic** is the point [*x*,*y*] where *x*, *y* are of type **real** with the convention that integers are automatically coerced to reals.

Two pictures *p* and *q* can be combined to form a single picture by use of the infix operators **^** and **&**. *p* **^** *q* defines the single picture formed by joining the last point of *p* to the first point of *q* by a straight line. *p* **&** *q* simply puts the two pictures *p* and *q* together as a single picture. Note that in both cases there is an implied ordering of the points in the resulting picture.

The algol-like constructs

shift *p* **by** *x*, *y*
scale *p* **by** *a*, *b*
rotate *p* **by** *theta*

perform the obvious transformations on a picture *p*. In particular,

scale *p* **by** 1, -1

reflects *p* in the *x*-axis and

rotate *p* **by** -90

rotates *p* about the origin by 90° in the anticlockwise direction.

As in other algol-like languages, these constructs may be nested so that

scale rotate *p* **by** -90 **by** -1,1

would first rotate *p* by 90 degrees in the anticlockwise direction and then reflect the resulting picture in the *y*-axis.

Objects of type **pic**, as all other data types, can be used as parameters of procedures or can be returned as the result of a procedure. Thus

procedure *thing* (**pic** *p*; **int** *m*, *n* → **pic**)

is the heading for a procedure which takes three parameters, the first being of type **pic** and the other two of type **int** and returns an object of type **pic**. A data type preceded by **c**, for example, **cpic**, implies that the parameter is a constant of the appropriate type.

Clearly, this language enables geometrical operations on pictures to be described exactly according to the geometrical transformations to be carried out. This enables very simple but powerful algorithms to be written and helps in understanding the

problem itself. This two-way communication between the programmer and the language contributed to the simplification of the following algorithms.

HILBERT CURVES

The three diagrams in Figure 1 illustrate the way in which the n th Hilbert polygon H_n is obtained from H_{n-1} . The polygon H_{n-1} is moved into the four corners of a larger square after suitable rotation and these four figures are joined by three line segments. Most previous algorithms require some scaling to be done at each iteration or for the basic step length to be determined before commencing the drawing of H_n . In the Outline solution a unit step length is used throughout to describe the successive iterates and all scaling is done at the final plotting phase by a simple choice of the plot screen limits.

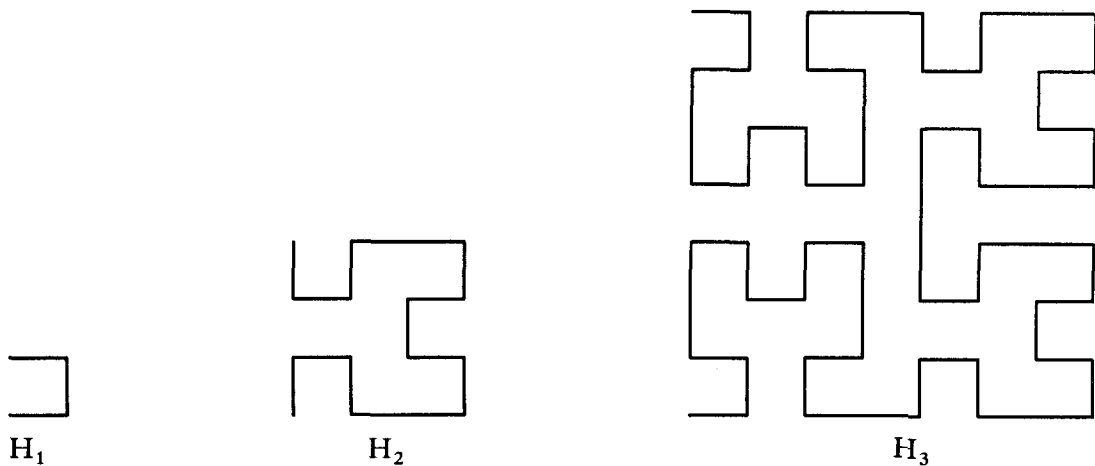


Figure 1. Hilbert polygons

Each Hilbert polygon starts in the bottom left-hand corner and finishes in the top left-hand corner. The bottom left-hand corner of H_n is made up of H_{n-1} rotated through -90 degrees and then reflected in the y -axis. The Outline clause to do this is

scale rotate h by -90 by $-1,1$

where h is the name of the abstract picture describing H_{n-1} . Similarly the bottom right hand corner of H_n is formed by shifting H_{n-1} in the positive x direction by a distance equal to the width of H_{n-1} plus 2. Note that 2 is indeed a constant so that the relative distance between components decreases as n increases. The Outline clause to do this is

shift h by $width + 2, 0$

Similarly, the Outline clauses to described the top right- and left-hand components of H_n are

shift h by $width + 2, width + 2$

and

shift scale rotate h by -90 by $1, -1$ by $width, 2 * width + 2$

It only remains to join these four components together using the $\hat{}$ operator. Since rotation and scaling of the origin $[0,0]$ leaves it unchanged the whole process can be started with H_0 being the single point $[0,0]$.

The complete procedure to describe and draw the Hilbert polygon H_n of complexity n is given in program 1. Note that this program requires no assignments.

```

procedure draw.hilbert( cint complexity )
begin
  procedure hilbert( cpic h ; cint order,width )
    if order = complexity then draw( h,0,width,0,width ) else
      hilbert( ( scale rotate h by -90 by -1,1  $\hat{\phantom{x}}$ 
        shift h by width + 2,0  $\hat{\phantom{x}}$ 
        shift h by width + 2,width + 2  $\hat{\phantom{x}}$ 
        shift scale rotate h by -90 by 1,-1 by width,
          2 * width + 2 ), order + 1,2 * width + 2 )
      hilbert( [ 0,0 ],0,0 )
    end

```

Program 1. Hilbert curves

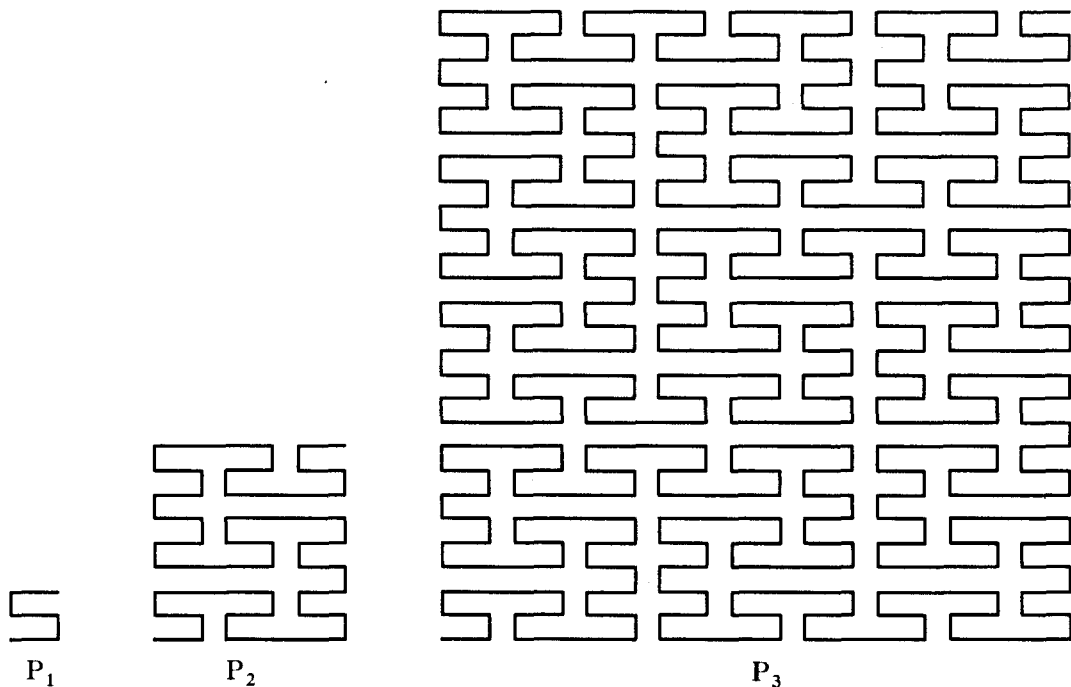


Figure 2. Peano polygons

PEANO CURVES

The Peano polygons shown in Figure 2 are not exact representations of successive approximations to the original Peano curve¹ but an obvious Peano analogue of the Hilbert construction. As shown above, the Hilbert construction for H_n is based upon four transformations of H_{n-1} . Similarly, the Peano curve P_n is normally regarded as consisting of nine transformations of P_{n-1} suitably placed and joined. However it is easy to see that the transformation from P_{n-1} to P_n can be broken down into two identical steps of three transformations each. The transformation from P_1 to P_2 is done in two steps with the first shown in Figure 3. Note that in order to get the orientation right it is necessary to rotate and reflect $P_{1.5}$ before using it in P_2 .

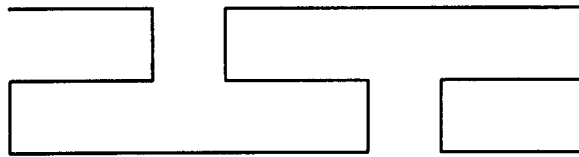


Figure 3. Peano polygon 1.5

It is not immediately obvious how to start the recursive calls of the procedure to draw the Peano polygons. One way would be to start with a line segment $[0,0] \hat{=} [2,0]$ but this is not so aesthetically pleasing or so clean as the single starting point $[0,0]$ for the Hilbert case. The confusion arises from the polygon P_1 which appears to have only six vertices rather than the nine which would lead to a more obvious solution. If however we imagine each of the three horizontal lines in P_1 to be broken by a vertex at its mid point then the generalization from the starting polygon P_0 consisting of the single point $[0,0]$ is obvious.

The complete procedure to describe and draw the Peano polygons is given in program 2.

```

procedure draw.peano( cint complexity )
begin
  procedure peano( cpic p ; cint complexity,order,old.width,width )
  if order = complexity then draw( p,0,width,0,width ) else
  peano( scale rotate ( p ^
    shift scale p by 1,-1 by old.width + 2,width ^
    shift p by 2 * old.width + 4,0 ) by -90 by -1,1,
    complexity,order + 1,width,3 * old.width + 4 )

  peano( [ 0,0 ],2 * complexity,0,0,0 )
end

```

Program 2. Peano Polygons

Peano noted that it is easy to generalize his result from the 3×3 repetitive pattern to an $n \times n$ pattern for any odd $n > 3$. Program 2 can be trivially modified to do this. It is also easy to modify the gaps between the components of the successive polygons to produce less regular pictures. Figures 4 and 5 show a wicket gate and a tartan effect, respectively, produced by irregular spacing of a 5×5 Peano polygon.

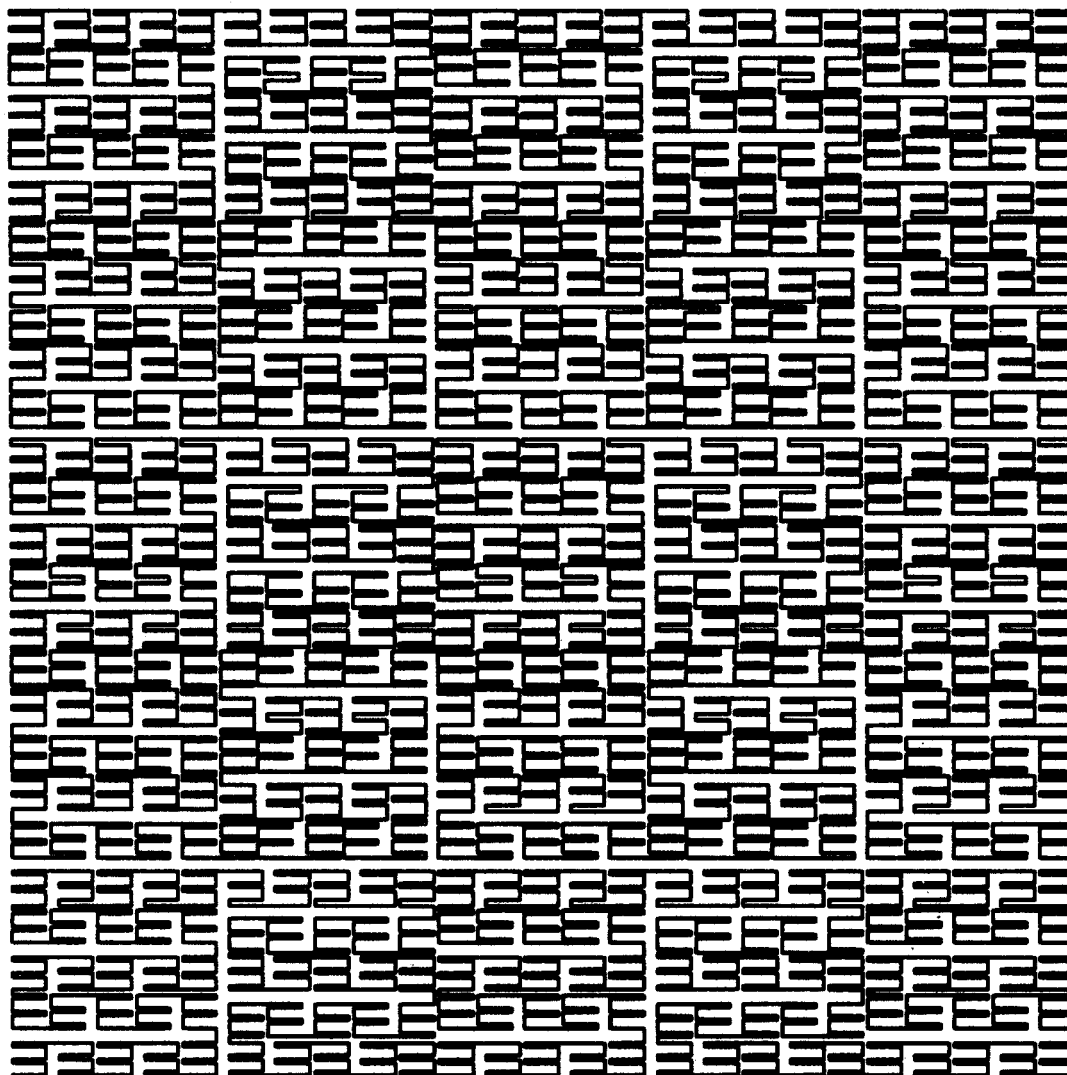


Figure 4. Wicket gate polygon

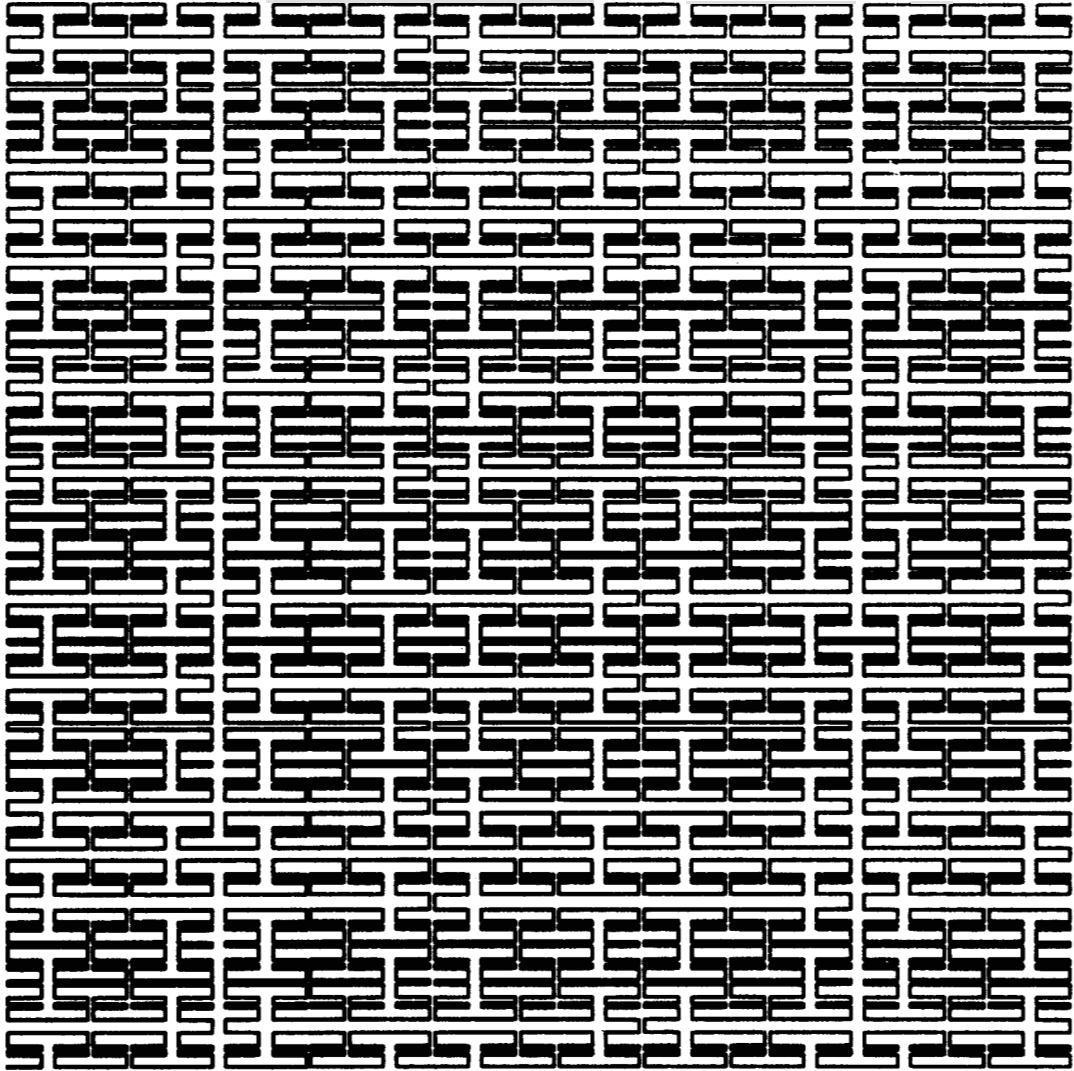


Figure 5. Tartan polygon

SIERPINSKI CURVES

The first few Sierpinski curves are shown in Figure 6. As Wirth points out these differ from the Hilbert and Peano polygons in that they are closed curves made up of four parts bounded by the principal diagonals and congruent by rotation about the origin. It is easier to see how to build up S_n if we consider only a quarter S'_n of each curve without the line segments joining it to the next quarter. S'_1 , S'_2 and S'_3 are shown in Figure 7. It is now obvious that S'_n is obtained from S'_{n-1} by suitably rotating and shifting S'_{n-1} to four new positions and joining them by three lines. S_n is then obtained by rotating S'_n four times and finally closing the last gap. Wirth suggests that S_0 is a square standing on one corner. The starting curve is thus the single point $[1,0]$.

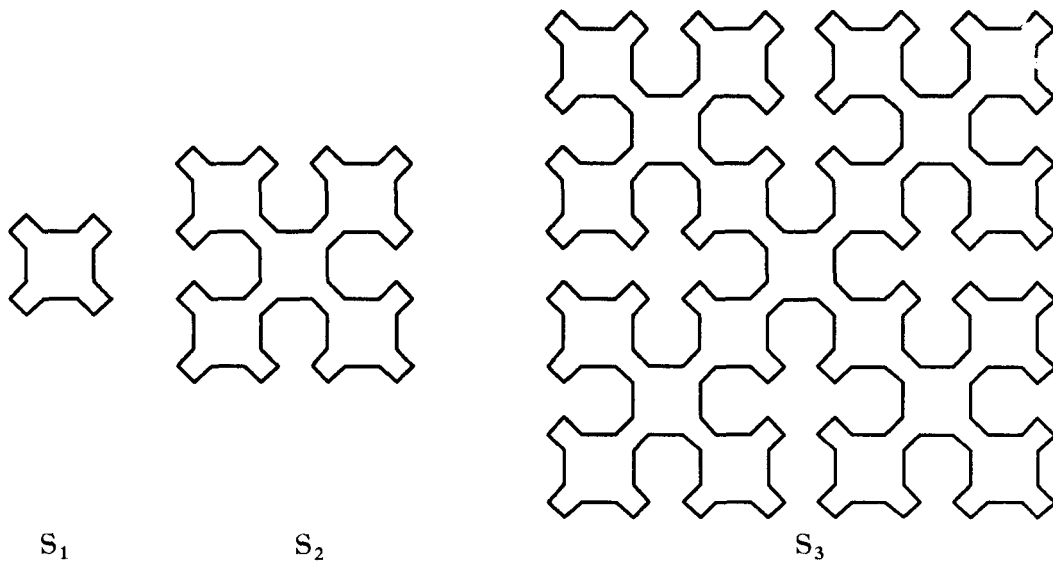


Figure 6. Sierpinski polygons

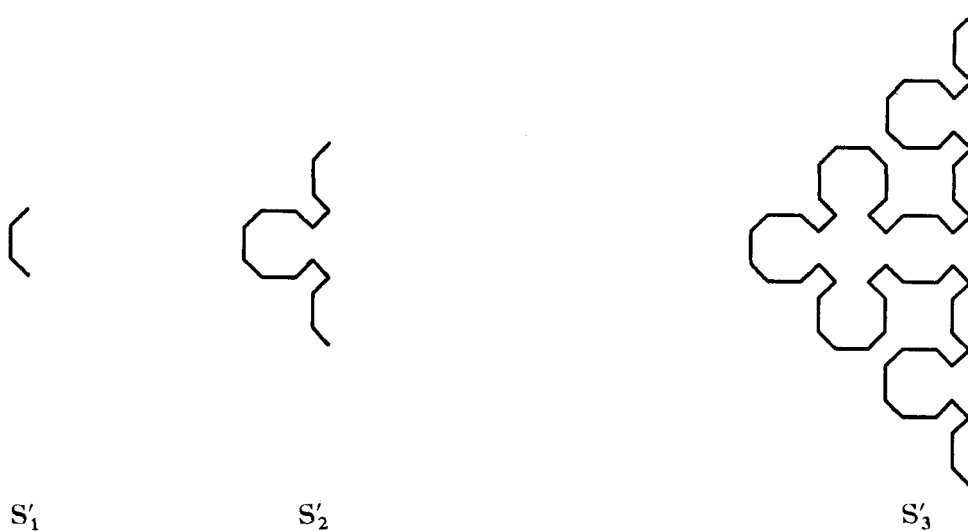


Figure 7. Basic Sierpinski polygons

The complete program to describe and draw the Sierpinski polygons is given in program 3. It is again easy to modify this program to draw more complex snowflake curves and also to increase the number of constituent segments from four to any chosen number.

```

procedure draw.sierpinski( cint complexity )
begin
  procedure rotate.and.draw( cpic s;cint width )
    draw( s ^ rotate s by -90 ^ rotate s by -180 ^
      rotate s by -270 ^
      [ width -1,-width + 2 ],-width,width,-width,width )

  procedure sierpinski( cpic s ; cint order,width )
    if order = complexity then rotate.and.draw( s,width ) else
      sierpinski( ( shift s by width,-width ^
        shift rotate s by -90 by width,-width ^
        shift rotate s by 90 by width,width ^
        shift s by width,width ),order + 1,2 * width )

  sierpinski( [1,0],0,2 )
end

```

Program 3. Sierpinski curves

CONCLUSION

The S-algol Outline graphics system has been used to describe and draw the Hilbert, Peano and Sierpinski curves. In each case the procedures have been written without assignment. In each case the starting point for the iterative procedures has been a single point. The close analogy between the program and the underlying geometry helped in arriving at this result.

REFERENCES

1. G. Peano, 'Sur une courbe, qui remplit toute une aire plane', *Math. Annln.*, **36**, 157-160 (1890).
2. D. Hilbert, 'Ueber stetige Abbildung einer Linie auf ein Flächenstück', *Math. Annln.* **38**, 459-460 (1891).
3. W. Sierpinski, 'Sur une nouvelle courbe qui remplit toute une aire plane'. *Bull. Acad. Sic. Cracovie, Serie A*, 462-478, (1912).
4. E. H., Moore, 'On certain crinkly curves', *Trans. Am. Math. Soc.*, **1**, 72-90 (1900).
5. A. Null 'Space filling curves, or how to waste time with a plotter'. *Software—Practice and Experience*, **1**, 403-410 (1971).
6. N Wirth, *Algorithms + data structures = programs*, Prentice-Hall, 1976.
7. L. M. Goldschlager, 'Short algorithms for space filling curves', *Software—practice and Experience*, **11**, 99-100 (1981).
8. I. H. Witten and B. Wyvill, 'On the generation and use of space-filling curves', *Software—practice and experience*, **13**, 519-525 (1983).
9. A. J. Cole and R. Morrison, *An Introduction to Programming with S-algol*, CUP, Cambridge 1982.
10. R. Morrison, 'Low cost computer graphics for micro computers', *Software—Practice and Experience*, **12**, 767-776 (1982).